Speech and Audio Recognition

Midterm Review

소리의 표본화부터 전사 없는 표현 학습까지 연결한다.

Midterm Review · Lecture 2–5
Signal → Spectrum → Alignment → Representation

Exam Map

각 장은 정보를 남기는 방식과 학습 target을 바꾼다.

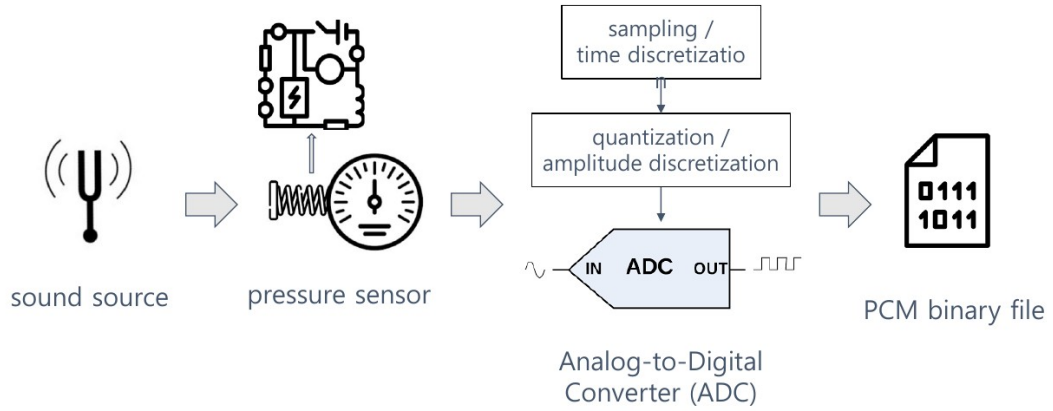
Stage	Input	Core question	Output
Digitize	pressure	언제·얼마나 측정?	PCM
Analyze	samples	어느 주파수가 언제?	STFT/Mel
Align	frames	token과 어떻게 대응?	CTC/LAS/RNN-T
Pre-train	unlabeled audio	target을 어디서 생성?	speech features

신호 처리에서는 보존 정보와 손실 조건을, ASR에서는 정렬과 출력 이력을 구분한다.
SSL에서는 transcript 없이 target을 만드는 방식을 비교한다.

Sampling and PCM

Sampling은 시간, quantization은 진폭을 이산화한다.

Analog and Digital Signals



The sinusoidal waveform follows from physics equations, see this slide
https://theory.labster.com/sound-waves-dbs/

5

Lecture 2, PDF p. 05 — analog pressure, sampling, quantization, PCM

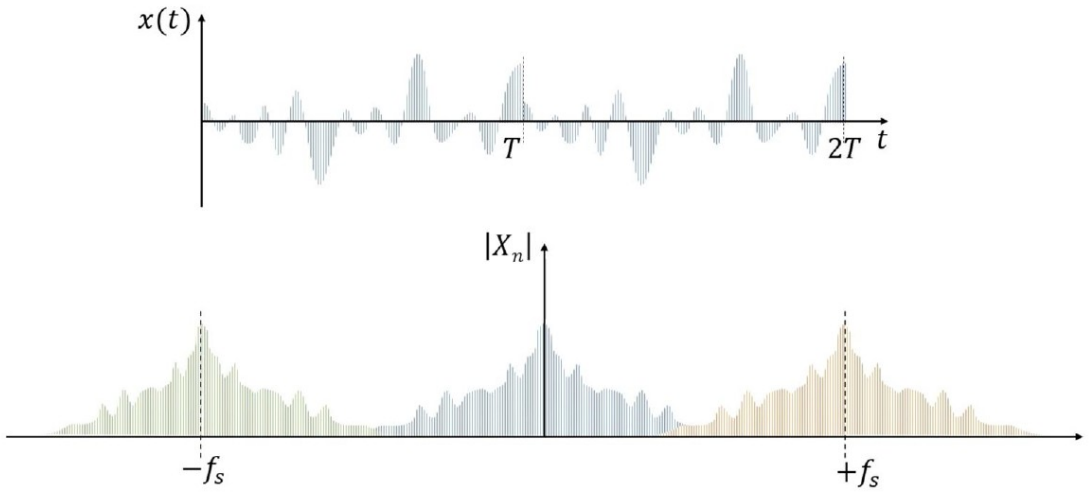
16-bit·44.1 kHz·stereo PCM의 payload는
$16 \times 44{,}100 \times 2 = 1{,}411{,}200$ bit/s다.

낮은 sampling rate와 낮은 bit depth는 서로 다른 정보를 잃는다.

Nyquist and Aliasing

Fourier basis는 표본의 주파수 성분을 분석한다. Aliasing 방지는 표본화 전에 결정된다.

Discrete-Time Fourier Series (DTFS)



What if the Nyquist sampling condition is not satisfied?

https://www.youtube.com/watch?v=nreiTseFZQ0

41

Lecture 2, PDF p. 37 — spectral replicas overlap when Nyquist is not satisfied

fs=8 kHz에서 zero-phase 5 kHz와 3 kHz cosine은 같은 표본을 만든다:
$\cos(2\pi \cdot 5n/8) = \cos(2\pi \cdot 3n/8)$.

표본화 전 anti-alias low-pass filter와 충분한 fs가 겹침을 막는다. FFT 길이나 window는 이미 잃은 원 주파수를 복원하지 못한다.

Orthogonal Fourier Bases

서로 다른 harmonic basis의 한 주기 inner product는 0이다.

Fourier Series: Exponential form

- Fourier Series & Fourier Coefficient
  - **Fourier Series:** exponential form

$$x(t) = \sum_{n=-\infty}^{\infty} c_n e^{2\pi i \frac{n}{P} t}$$

Frequency component (Frequency basis)

pure frequency component at $n\omega_0$
n=0 → 0 Hz (DC)
n=1 → f_0
n=2 → $2f_0$
…

- $\omega_0 = \frac{2\pi}{P}$
- Frequency basis:
  $\cdots, e^{-j2\omega_0 t}, e^{-j\omega_0 t}, 1, e^{j\omega_0 t}, e^{j2\omega_0 t}, \cdots$
- What is the basis?

$$e_1 = \begin{bmatrix}1\\0\\0\end{bmatrix} \quad e_2 = \begin{bmatrix}0\\1\\0\end{bmatrix} \quad e_3 = \begin{bmatrix}0\\0\\1\end{bmatrix}$$

$e_i^T e_i = 0 \ (i \neq j)$ — Orthogonal basis

- Frequency basis:

Inner product

$$\phi_n(t) = e^{jn\omega_0 t}$$

$$< \phi_n, \phi_m > = \int_0^P \phi_n(t)\phi_m^*(t)dt$$

$$= \int_0^P e^{jn\omega_0 t} e^{jm\omega_0 t} dt = \int_0^P e^{j(n-m)\omega_0 t} dt$$

$$< \phi_n, \phi_m > = \begin{cases} P, n \neq m \\ 0, n = m \end{cases}$$

28

Lecture 2, PDF p. 26 — complex exponential bases and one-period inner product

직교성은 inner product로 각 주파수 성분을 분리할 수 있다는 뜻이다.

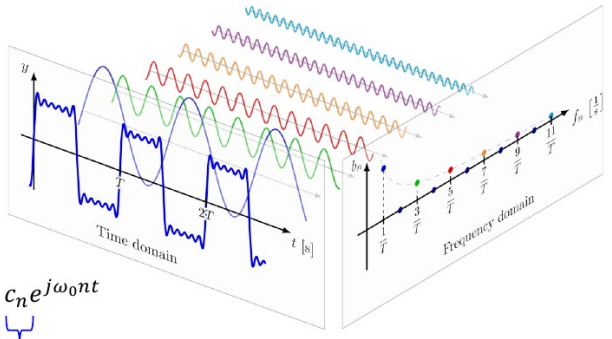
원문 빨간 식의 조건이 뒤바뀌었다: n=m이면 P, n≠m이면 0이다.

Fourier coefficient는 각 basis 방향의 성분을 측정한다.

Series to Transform

주기가 커지면 harmonic 간격이 좁아져 합이 적분으로 이어진다.

Fourier Transform



- Fourier Series & Fourier Coefficient

P: the period

$$c_n = \frac{1}{P}\int_{-P/2}^{P/2} x(t)e^{-2\pi i\frac{n}{P}t}dt \qquad x(t) = \sum_{n=-\infty}^{\infty} c_n e^{j\omega_0 nt}$$

Fourier coefficient

$$\omega_0 = \frac{2\pi}{P} \qquad \frac{1}{P} = \frac{\Delta\omega}{2\pi}$$

- Let's assume $P \to \infty \rightarrow \omega_0 \to 0 \rightarrow \Delta\omega = \omega_0 \to 0$

Frequency — Original signal

$$X(\omega) = \int_{-\infty}^{\infty} x(t)e^{-j\omega t}dt$$

Fourier transform

- Fourier Transform!
  - A mathematical formular that allows us to decompose any signal into its individual **frequencies** and the frequency's **amplitude**

https://en.wikipedia.org/wiki/Fourier_series

32

Lecture 2, PDF p. 29 — increasing period turns frequency sums into an integral

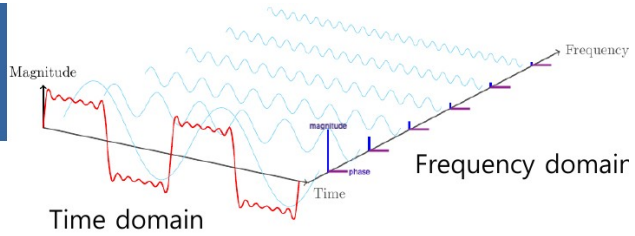
Fourier series는 주기 신호의 이산 harmonic 계수를, Fourier transform은 비주기 신호의 연속 spectrum을 표현한다.

정변환은 성분을 분석하고
역변환은 그 성분을 합성한다.

CTFS to DTFS

한 주기를 N번 표본화하면 harmonic index가 N을 주기로 같아진다.

Fourier Series → DTFS



- Fourier Series & Fourier Coefficient

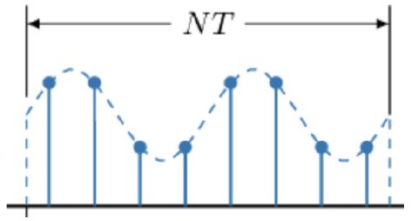
Fourier Series

$$x(t) = \sum_{n=-\infty}^{\infty} c_n e^{2\pi i \frac{n}{P} t}$$

Fourier coefficient

$$c_n = \frac{1}{P} \int_{-P/2}^{P/2} x(t) e^{-2\pi i \frac{n}{P} t} dt, \qquad \phi_k(t) = e^{jk\omega_0 t}$$

- Discrete-Time Fourier Series
  - In practice our time signal is time-limited and contains N non-zero samples taken with a sampling period T seconds.
  - Let's explore what happens when we try to find its Fourier coefficients.



https://ru.dsplib.org/content/dft/dft.html

36

Lecture 2, PDF p. 32 — periodic continuous signal and N uniform samples

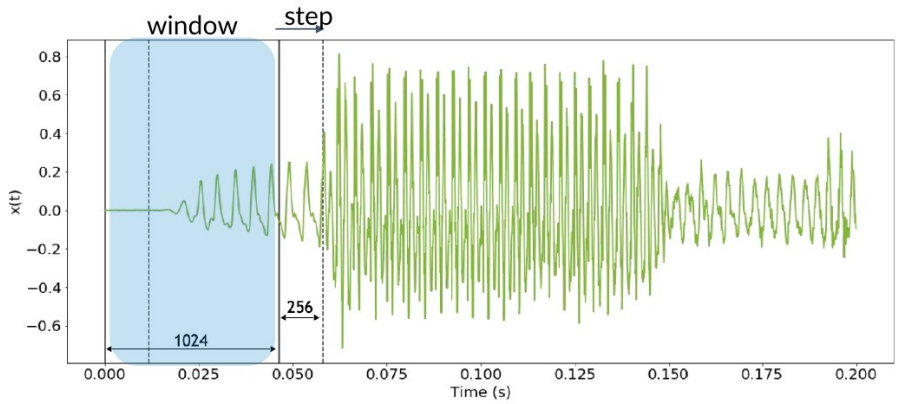
k와 k+qN은 표본 위에서 같은 basis이므로 CTFS 계수가 같은 나머지 class로 합쳐진다.

N〉2K와 제한된 harmonic bandwidth가 있어야 원래 계수를 겹침 없이 회수할 수 있다.

STFT Framing

STFT는 짧은 창을 이동하며 각 구간의 spectrum을 계산한다.

Shor-Time Fourier Transform



16 kHz, L=400, H=160은 25 ms 창과 10 ms hop이다.

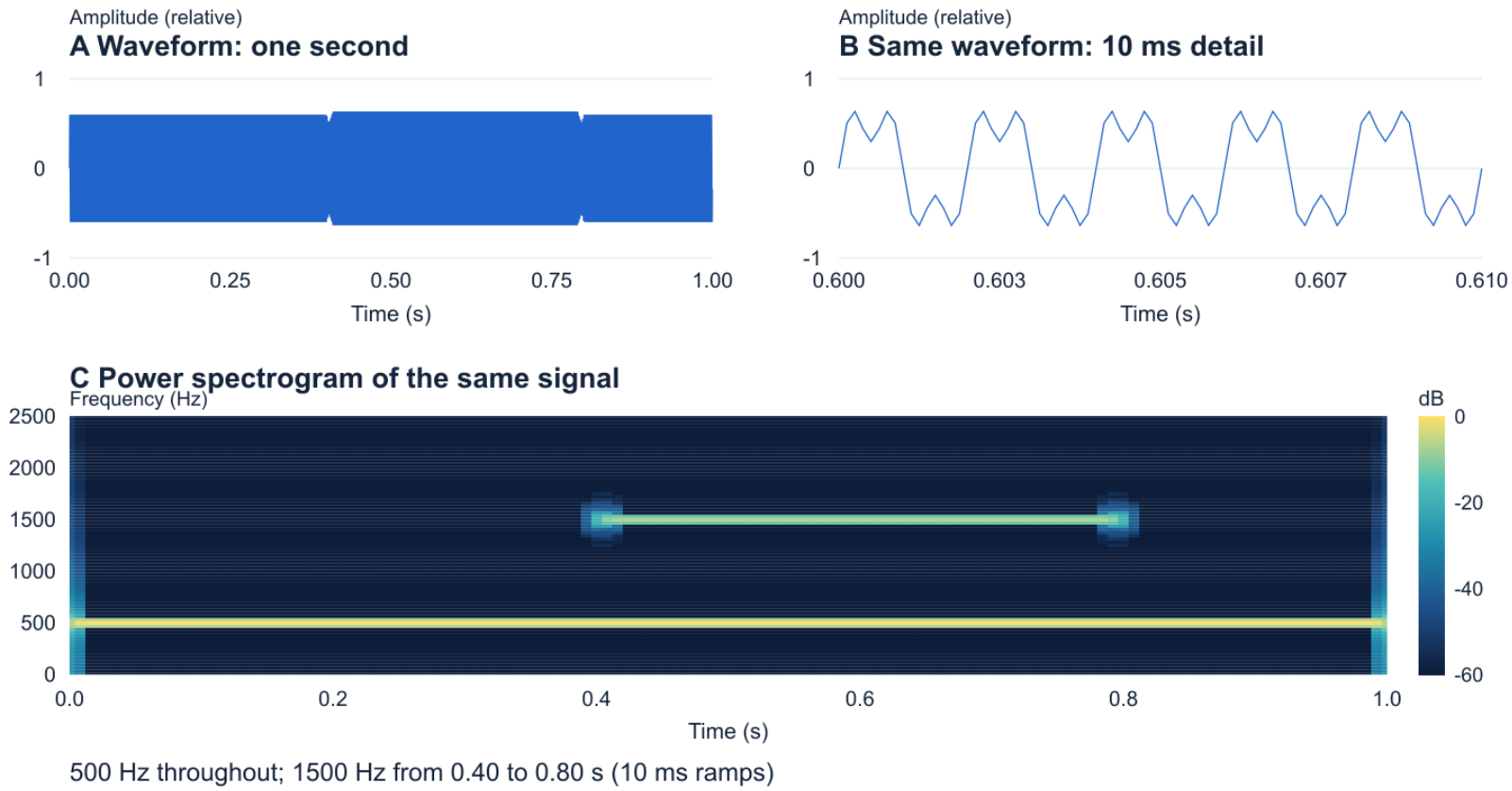
1초 신호를 center padding 없이 처리하면 98 frame, NFFT=512의 one-sided spectrum은 257 bin이다.

9

Lecture 2 Lab, PDF p. 09 — moving analysis window and overlap

Waveform and Spectrogram

파형은 시간별 진폭을, spectrogram은 시간별 주파수 구성을 보여준다.



작성자 계산 예제: 8 kHz, Hann 32 ms, hop 8 ms, NFFT 256. 색은 최대 power 기준 dB이며 위상은 표시하지 않는다.

Magnitude and Phase

복소 STFT에는 크기와 위상이 함께 있다. 일반적인 spectrogram은 크기나 power만 색으로 표시한다.

X = 3 + 4j와 3 – 4j는 모두 크기 5, power 25다. 위상은 각각 +53.13° 와 −53.13° 이므로 색만으로 둘을 구분할 수 없다.

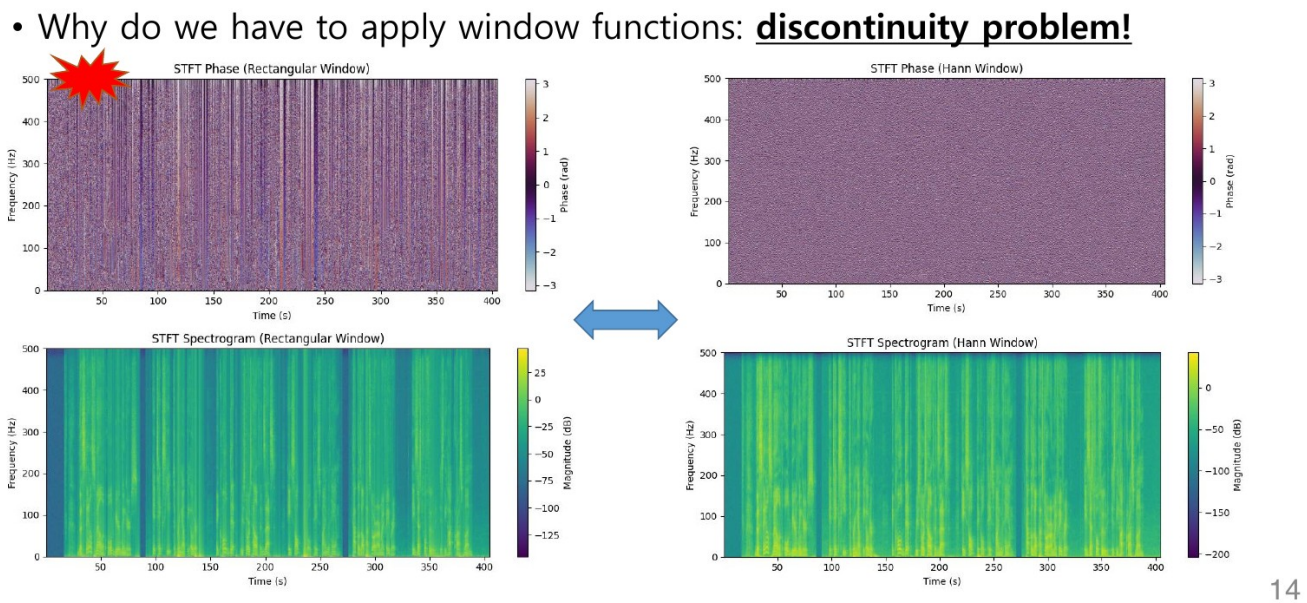
실수부와 허수부가 함께 크기와 위상을 결정한다. 원신호 x와 −x도 power spectrogram은 같지만 파형의 부호는 반대다.

정확한 역변환에는 complex STFT와 분석 설정, 원구간 전체를 덮는 window 및 0이 아닌 overlap 가중치가 필요하다. 그림만으로 원래 파형을 유일하게 정할 수 없다.

Window Trade-offs

Window는 leakage를 없애지 않고 main-lobe와 side-lobe의 절충을 바꾼다.

Shor-Time Fourier Transform



Rectangular window는 경계 불연속 때문에 먼 sidelobe가 크다.

Hann window는 먼 leakage를 낮추지만 peak를 더 넓게 보이게 할 수 있다. 색 범위가 다르면 그림 밝기를 직접 비교하지 않는다.

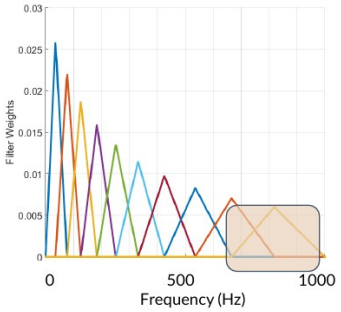
Lecture 2 Lab, PDF p. 14 — rectangular and Hann STFT views

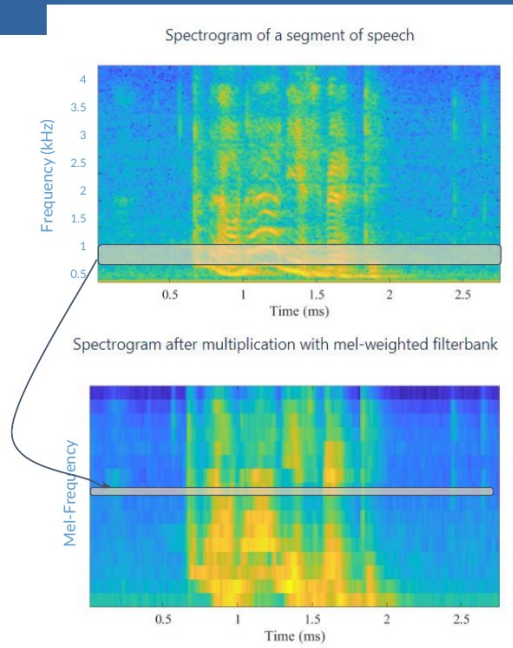
Mel Compression

Mel filterbank와 log는 서로 다른 두 압축이다.

Mel Spectrogram

- Mel Spectrogram





https://wiki.aalto.fi/display/ITSP/Cepstrum+and+MFCC

17

Filterbank는 여러 Hz bin을 band 별로 합쳐 주파수축을 줄인다. Log는 값의 dynamic range를 줄인다.

인식에는 유용하지만 phase와 세밀한 bin 정보를 잃으므로 원래 complex STFT를 일반적으로 복원할 수 없다.

Lecture 2 Lab, PDF p. 17 — triangular Mel filters and Mel spectrogram

Word Error Rate

WER은 편집 오류 수를 정답 단어 수로 정규화한다.

Symbol	Meaning	Example count
S	substitution	1
D	deletion	1
I	insertion	1
N	reference words	4

Reference 4단어에서 substitution 1, deletion 1, insertion 1이면 WER=(1+1+1)/4=0.75다.
Insertion이 많으면 WER은 1을 넘을 수 있다. 빈 reference에서는 별도 평가 규칙이 필요하다.

Byte Pair Encoding

BPE는 corpus에서 자주 붙는 token pair를 반복 병합한다.

DL-based ASR

- Subword Tokenization: BPE (Byte Pair Encoding)

Algorithm 1 Byte-pair encoding [30]

Input: set of strings D, target vocab size K
procedure BPE(D, K)
  $V \leftarrow$ all unique characters in D
  while $|V| < K$ **do** ▷ *Merge tokens*
    $t_L, t_r \leftarrow$ Most frequent bigram in D
    $t_{NEW} \leftarrow t_L + t_R$ ▷ *Make new token*
    $V \leftarrow V + [t_{NEW}]$
    Replace each occurrence of t_L, t_r in D with t_{NEW}
  end while
  return V
end procedure

aaabdaaabac

1. add Z <- aa
   ZabdZabac

2. add Y <- ab
   ZYdZYac

3. add X <- ZY
   XdXac

Example: low, lowest, newer → add "lo"=**A**, **A**w, **A**west, newer → ...
V: l, o, w, n, e, s, t, r — **V**: **A**, w, n, e, s, t, r

47

Lecture 3, PDF p. 47 — BPE merge algorithm and toy sequence

Training은 merge 순서를 만들고 저장한다. Inference는 새 문장에 같은 순서를 적용한다.

Subword는 출력 길이 U를 줄일 수 있지만 vocabulary V와 softmax 비용은 커질 수 있다.

CTC Blank and Paths

CTC blank는 frame 정렬을 허용하고 최종 전사에서는 제거된다.

Path	Merge repeats	Drop blank	Output
L ε L	L ε L	LL	LL
L L	L	L	L
ε a b	ε a b	ab	ab
a a b	a b	ab	ab

Collapse는 연속 중복을 먼저 합친 뒤 blank를 제거한다.
따라서 L ε L은 LL이지만 L L은 L이다. 같은 label을 두 번 출력하려면 사이 blank가 필요하다.

CTC Probability

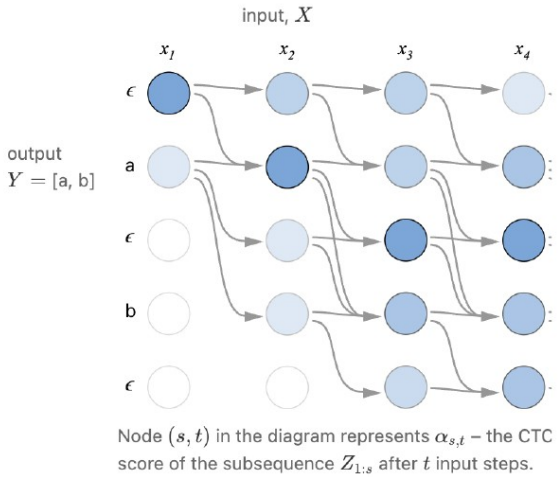
한 문자열의 확률은 그 문자열로 collapse되는 모든 경로 확률의 합이다.

DL-based ASR

t	t=1	t=2	t=3	t=4
ε	0.6	0.2	0.3	0.4
a	0.3	0.5	0.2	0.1
b	0.1	0.3	0.5	0.5

O: probability distribution

- Efficient CTC Loss Compute: Example
  - Output $Y = [a, b]$



Node (s, t) in the diagram represents $\alpha_{s,t}$ – the CTC score of the subsequence $Z_{1:s}$ after t input steps.

33

각 frame의 분포는 encoder logit의 softmax에서 나온다. 한 경로 확률은 선택한 frame 확률의 곱이다.

서로 배타적인 유효 경로를 더한 뒤 −log를 취해 CTC loss를 얻는다.

Lecture 3, PDF p. 33 — frame distributions and valid CTC paths

CTC Forward DP

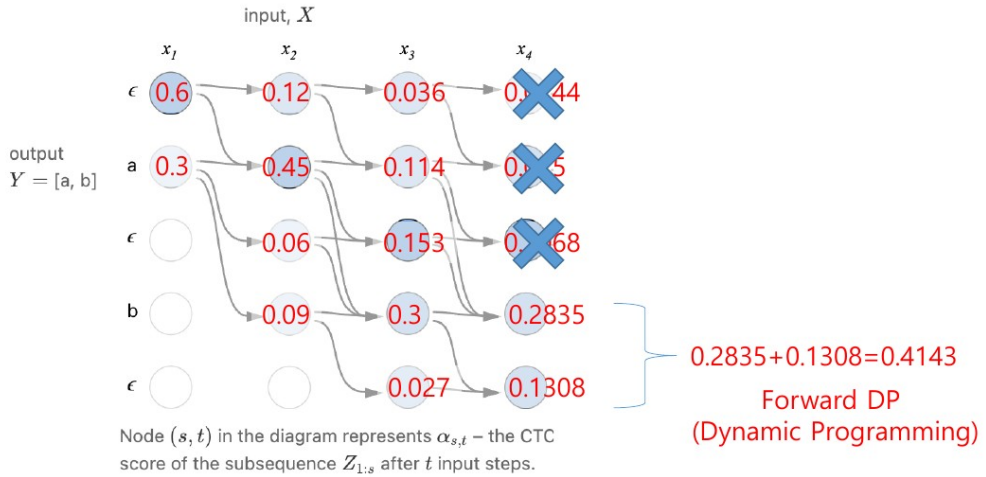
각 칸은 해당 상태에서 끝나는 모든 유효 부분 경로의 확률 합이다.

DL-based ASR

t	t=1	t=2	t=3	t=4
ε	0.6	0.2	0.3	0.4
a	0.3	0.5	0.2	0.1
b	0.1	0.3	0.5	0.5

O: probability distribution

- Efficient CTC Loss Compute: Example
  - Output $Y = [a, b]$



Node (s, t) in the diagram represents $\alpha_{s,t}$ – the CTC score of the subsequence $Z_{1:s}$ after t input steps.

35

Lecture 3, PDF p. 35 — forward probabilities and two terminal states

Stay·한 칸 전진·허용된 두 칸 skip 의 이전 합에 현재 frame 확률을 곱한다.

P(ab|X)=0.2835+0.1308=0.4143. 유한 경로의 합이며 극한 수렴이 아니다.

원문 합산식의 0.1038은 0.1308 의 오기다.

CTC Summary: The Whole Process

CTC는 정답의 발음 시각을 지정하지 않고 문자열 전체의 확률을 학습한다.

DL-based ASR

- CTC Summary

* Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks, A. Graves et. al, 2006

CTC Properties

- conditional independence
- streamable
- produces alignment
- fast inference

CTC Decoding

- allows to reduce fixed len predictions to variable length predictions
- provides mapping from path to string

How to train:

- compute encoder predictions - output matrix
- efficiently compute sum of probs of all paths corresponding to target
- maximize this sum

How to evaluate:

- compute encoder predictions - output matrix
- argmax / beam search
- CTC decode

Beam Search:

- expand beam
- merge paths
- truncate beam
- repeat

41

왼쪽은 모델의 가정과 경로→ 문자열 변환이다. 오른쪽은 학습, 평가, beam 탐색의 순서다.

학습은 정답에 이르는 경로를 합한다. 추론은 정답을 모른 채 출력 후보를 찾는다. 둘은 같은 연산이 아니다.

Lecture 3, PDF p. 41 — CTC Summary; Korean walkthrough continues on the next slides

CTC Summary : Assumptions

조건부 독립은 음향 문맥이 없다는 뜻이 아니라 출력 이력을 직접 조건으로 쓰지 않는다는 뜻이다.

P(경로|X)는 frame별 확률의 곱이다. 각 분포는 encoder가 읽은 X의 문맥에 의존하지만, 앞서 고른 문자를 다시 입력받지는 않는다.

Blank와 반복 병합으로 긴 frame 열을 짧은 문자열로 바꾼다. 학습 중 정렬을 잠재 경로로 다루므로 문자별 시작·종료 시각 정답이 필요 없다.

Streaming은 미래 전체를 기다리지 않는 encoder가 있을 때 가능하다. CTC를 쓴다는 사실만으로 양방향 encoder까지 실시간이 되지는 않는다.

CTC Summary: Train and Decode

Loss를 계산할 때는 정답이 주어지고, decoding할 때는 정답을 찾아야 한다.

학습: encoder → frame별 softmax → 정답으로 collapse되는 경로의 합 $P(y|X)$ → $-\ln P(y|X)$. Forward DP가 이 합을 효율적으로 구한다.

추론: greedy는 frame별 최대값을 고르고 반복 병합·blank 제거를 한다. Prefix beam은 여러 문자열 후보의 경로 확률을 누적한다.

Beam은 확장 → 같은 prefix의 확률 병합 → 상위 B개 유지 순으로 반복한다. B가 커져도 모델 자체의 인식 오류까지 사라지지는 않는다.

CTC: Best Path vs. Best Text

두 frame 모두 P(ε)=0.5, P(a)=0.4, P(b)=0.1이라고 하자.

Path	Probability	Collapsed text
ε ε	0.5 × 0.5 = 0.25	빈 문자열
a a	0.4 × 0.4 = 0.16	a
a ε	0.4 × 0.5 = 0.20	a
ε a	0.5 × 0.4 = 0.20	a

주요 경로 4개를 표시했다. Greedy는 빈 문자열을 내지만 P(a|X)=0.16+0.20+0.20=0.56이다.
나머지 출력은 P(b)=0.11, P(ab)=P(ba)=0.04다. 다섯 출력의 확률 합은 1이다.

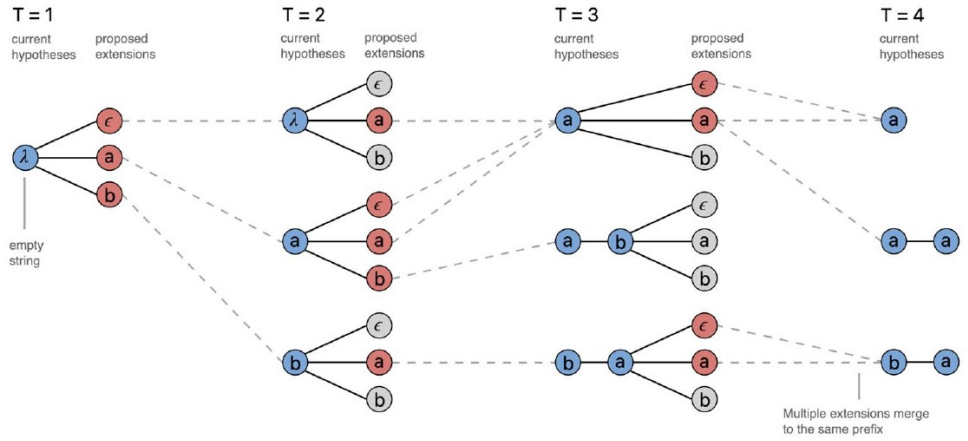
Greedy vs. Beam Search

최대 확률 경로가 최대 확률 문자열과 같다는 보장은 없다.

DL-based ASR

- CTC Beam Search (Beam size=3)

Beam search step:
- expand beam
- merge paths
- truncate beam



The CTC beam search algorithm with an output alphabet $\{\epsilon, a, b\}$ and a beam size of three.

40

Lecture 3, PDF p. 40 — prefix beam expansion and pruning

Greedy는 frame마다 한 token만 남긴다. Prefix beam search는 여러 prefix의 blank·non-blank 확률을 따로 누적한다.

유한 beam은 낮은 후보를 버리는 근사이며, 외부 LM을 쓰면 점수 scale과 length bonus도 확인한다.

Deep Speech 2 vs. Conformer

Encoder 구조와 정렬 loss는 같은 선택 축이 아니다.

Model	Local pattern	Long context	Objective / role
Deep Speech 2	convolution	recurrent layers	CTC objective
Conformer	conv module	self-attention	encoder architecture
CTC	—	—	path marginalization

Deep Speech 2는 convolution과 recurrent layers로 음향 표현을 만들고 CTC로 정렬한다.
Conformer는 self-attention의 넓은 문맥과 convolution의 국소 패턴을 한 encoder block에 결합한다. CTC와 함께 쓸 수 있다.

Encoders: What Changed?

Deep Speech 2와 Conformer의 차이는 음향 문맥을 표현하는 방식에 있다.

Deep Speech 2는 convolution으로 음향 패턴을 추출하고 recurrent layer로 시간 문맥을 전달한다. CTC는 그 표현에서 정렬 없는 문자열 학습을 맡는다.

Conformer는 self-attention의 넓은 문맥과 convolution의 가까운 시간 패턴을 함께 활용한다. 가까운 발음 변화와 멀리 떨어진 단서가 모두 필요하기 때문이다.

Encoder를 Conformer로 바꿔도 loss가 자동으로 정해지지는 않는다. CTC, attention decoder, RNN-T와의 결합은 별도 설계 선택이다.

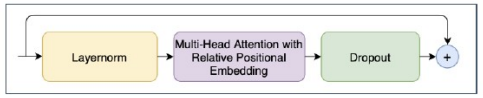
Conformer: Follow the Block

오른쪽 block은 아래에서 위로 읽는다. Attention 앞뒤를 feed-forward 층이 감싼다.

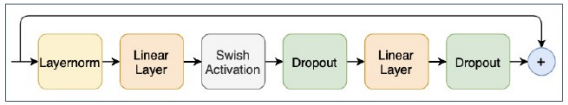
DL-based ASR

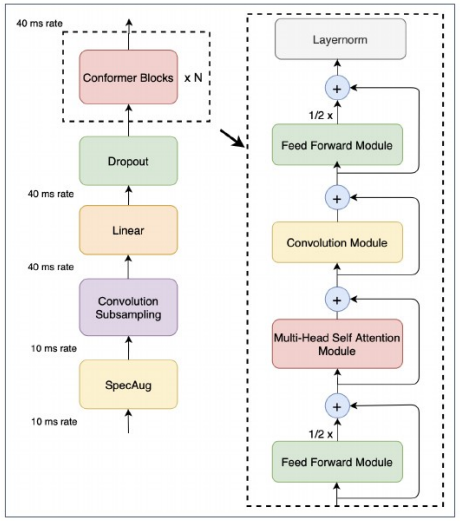
- Encoders: Conformer
  - **Conformer: Convolution-augmented Transformer for Speech Recognition, Anmol Gulati, Google, 2020**
    - Transformers
    - 1D depthwise separable convolution
    - convolutions for local, attention for global dependencies

Multi-head self-attention model



Feed forward module





43

½ FFN → self-attention → convolution → ½ FFN → layer normalization 순이다. 각 잔차 경로는 입력을 변환 결과에 더한다.

½은 FFN의 잔차 기여를 조절하는 계수다. 차원을 절반으로 줄인다는 뜻이 아니다. 시간 길이 축소는 왼쪽 subsampling이 맡는다.

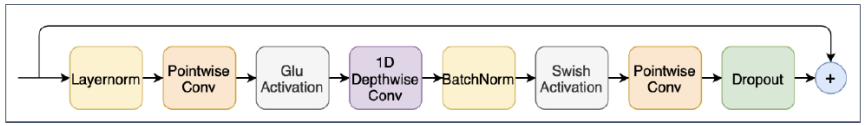
Lecture 3, PDF p. 43 — Conformer encoder and Macaron-style block

Conformer Convolution

Depthwise convolution은 채널별 시간 이웃을 처리하고 pointwise convolution은 채널을 섞는다.

DL-based ASR

- Encoders: Conformer
  - **Conformer: Convolution-augmented Transformer for Speech Recognition, Anmol Gulati, Google, 2020**



Convolution module

GLU가 전달할 채널 정보를 조절하고 residual이 입력과 변환 결과를 합친다.

Subsampling stride 2를 두 번 쓰면 시간 길이는 대략 T/4지만 kernel·padding에 따라 정확한 길이는 달라진다.

Lecture 3, PDF p. 44 — Conformer convolution module

Subword and Subsampling

Subword는 정답 길이 U를, subsampling은 입력 표현 길이 T를 줄인다.

BPE는 학습 corpus에서 자주 붙는 token 쌍을 차례로 병합한다. 추론은 이미 정해진 병합 규칙을 적용한다. 단어 전체 사전보다 미지 단어를 분해하기 쉽다.

Subsampling은 1D stride convolution, 2D convolution·projection, frame 묶기 ·projection 등으로 시간축을 줄인다. Stride 2 두 번이면 대략 T/4다.

CTC에는 $T \geq U + R$이 필요하다. R은 이웃한 동일 정답 token 쌍의 수다. 'LL'은 blank가 필요해 최소 3 frame이다. 입력을 과도하게 줄이면 정렬이 불가능하다.

From CTC to Attention

Attention은 다음 token을 만들 때 필요한 입력 위치를 다시 선택한다.

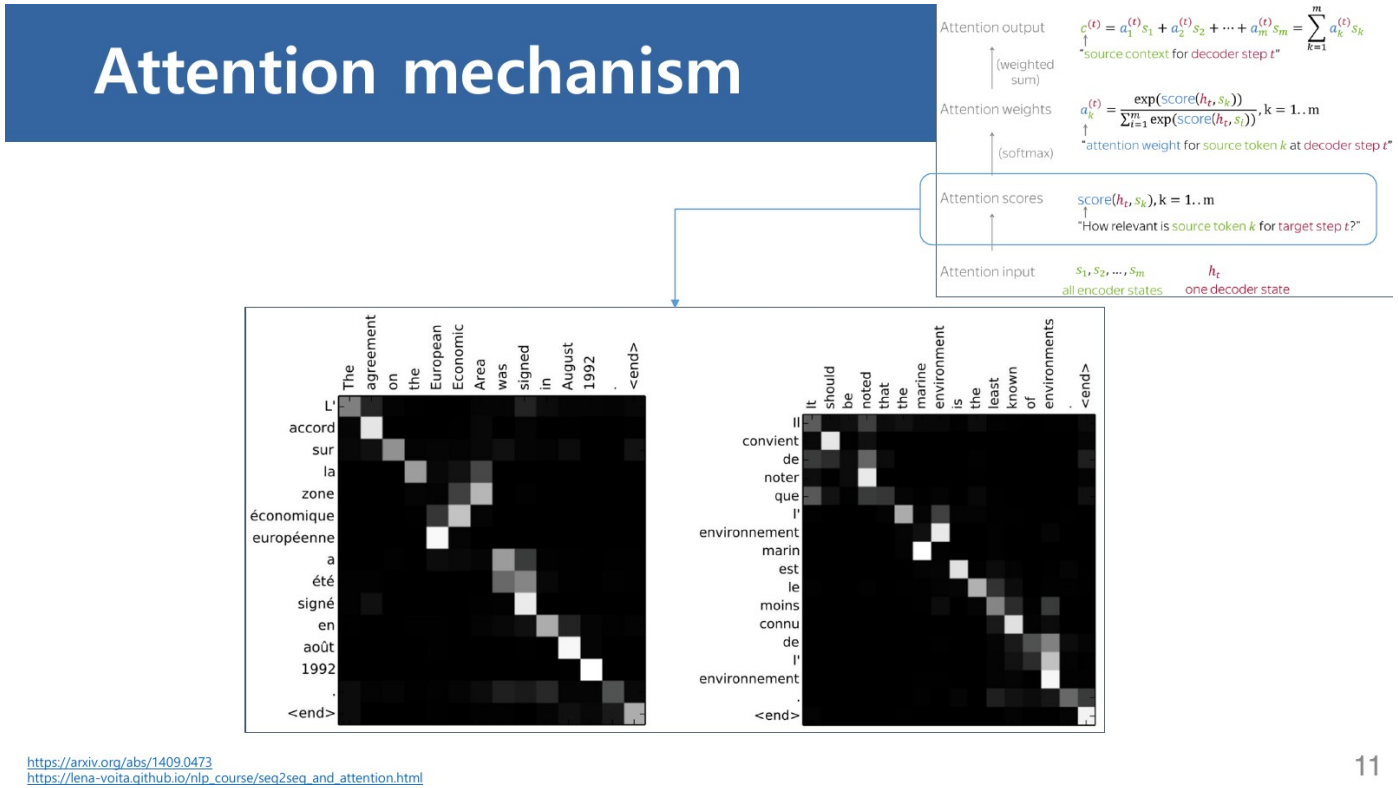
한 개의 고정 벡터에 긴 입력을 모두 압축하면, 출력 단계별로 필요한 정보가 달라도 같은 요약을 써야 한다. Attention은 encoder의 위치별 표현을 남겨 둔다.

Decoder의 현재 상태와 각 입력 표현의 관련도를 계산한다. Softmax로 가중치를 만들고, 입력 표현들의 가중합을 현재 context로 사용한다.

다음 출력 단계에서는 상태가 바뀌므로 가중치도 다시 계산한다. 번역의 단어 순서 변화와 음성 frame–문자 길이 차이를 유연하게 다룰 수 있다.

Attention: Read the Heatmap

각 행은 한 출력 단계가 어느 입력 위치에 높은 가중치를 주는지 보여 준다.



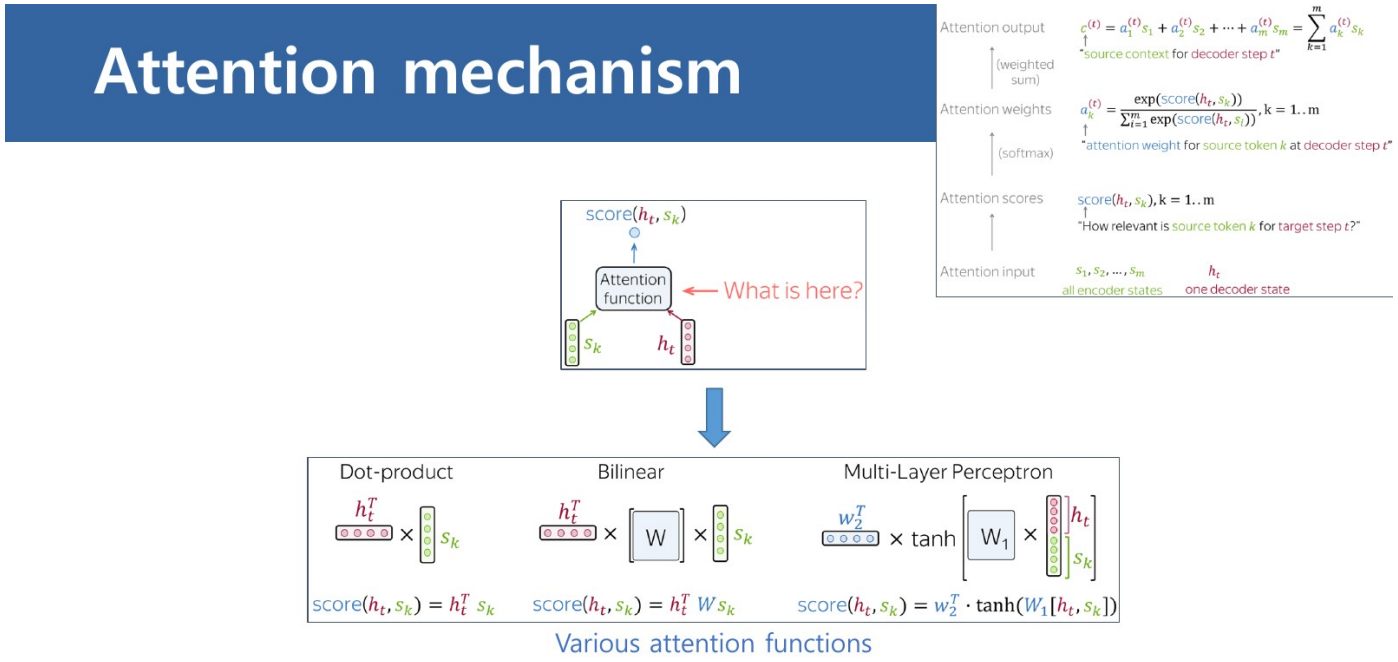
Lecture 4, PDF p. 11 — source-position softmax and input–output attention maps

가로는 입력 영어 token, 세로는 출력 프랑스어 token이다. 밝은 칸일수록 해당 출력이 그 입력에 큰 attention을 준다.

밝은 칸이 항상 대각선에 있지는 않다. 어순이 달라질 수 있기 때문이다. 이 가중치는 학습된 참조 패턴이지 정답 정렬의 증명은 아니다.

Attention: Three Scoring Functions

관련도 score를 계산하는 함수는 바뀌어도 softmax와 가중합의 역할은 같다.



https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

10

Lecture 4, PDF p. 10 — dot product, bilinear score, and an MLP score

Dot product는 두 표현을 바로 내적한다. Bilinear는 학습 행렬 W를 사이에 두어 어떤 성분을 비교할지 배운다.

MLP는 두 표현을 결합하고 비선형 함수를 거쳐 점수를 만든다. 세 방식 모두 출력은 아직 확률이 아닌 scalar score다.

Attention: Read the Numbers

Attention 가중치는 입력 위치에 대한 분포이며, 출력 단어 확률과 다르다.

관련도 $e(u,t) \rightarrow \alpha(u,t) = \exp(e(u,t)) / \Sigma_j \exp(e(u,j)) \rightarrow c(u) = \Sigma_t \alpha(u,t) h(t)$. 합이 1인 α 로 입력 벡터를 섞는다.

예: $h_1=(1,2)$, $h_2=(2,0)$, $h_3=(0,4)$, $\alpha=(0.2,0.3,0.5)$이면 $c=(0.8,2.4)$다. 세 번째 입력이 가장 큰 비중을 갖는다.

Decoder는 c와 이전 token 이력으로 어휘 확률을 계산한다. $\alpha_3=0.5$라는 사실은 세 번째 단어를 50% 확률로 출력한다는 뜻이 아니다.

Autoregressive Prediction

앞서 나온 token이 다음 token의 확률 분포를 바꾼다.

$P(y|X)=\pi_u P(y_u|y_{\langle u\rangle},X)$. EOS도 예측한다. CTC는 X가 주어졌을 때 frame별 경로 기호를 독립으로 두지만, 여기서는 출력 prefix를 조건으로 쓴다.

Teacher forcing 학습은 정답 prefix를 넣는다. 추론에서는 모델이 생성한 prefix를 넣으므로 앞선 오답이 다음 예측을 흔들 수 있다.

학습 loss는 각 단계의 정답 token에 대한 −ln 확률을 합한다. 이는 하나의 출력 순서에 대한 계산이며 CTC의 여러 frame 정렬 경로 합과 다르다.

Cross Entropy: Why −log?

정답에 큰 확률을 줄수록 손실이 작아진다.

어휘가 cat·dog·bird이고 정답이 dog라면 p=(0,1,0)이다. q=(0.1,0.7,0.2)일 때 CE = $-\Sigma\ p(i)\ \ln q(i) = -\ln(0.7) \approx 0.357$이다.

정답 확률이 0.2로 낮아지면 loss≈1.609로 커진다. 문장에서는 정답 확률들을 곱하는 대신 -log를 취해 단계별 loss를 더한다.

원문의 'dot'은 vocabulary와 one-hot을 보면 'dog'의 오기다. Cross entropy는 대칭적 거리라기보다 정답 분포에 대한 예측의 로그 손실이다.

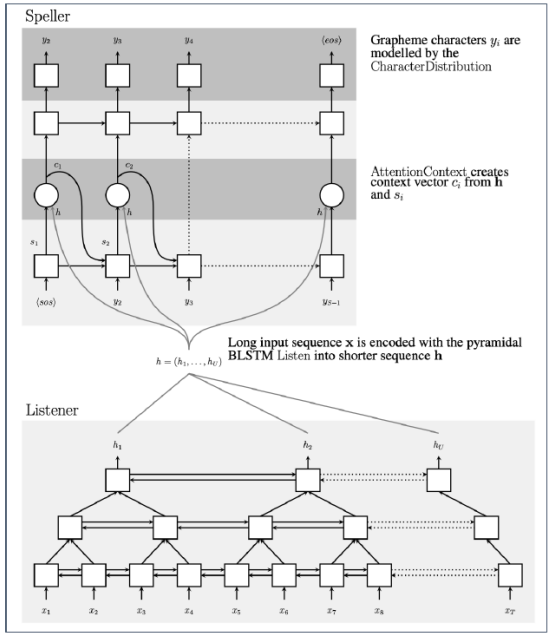
Listen, Attend and Spell

LAS는 출력 단계마다 필요한 입력 위치를 attention으로 다시 고른다.

Listen, Attend and Spell (LAS)

- **Listen, Attend and Spell**, William Chan et al., Carnegie Mellon University, Google Brain, 2015
  - The network produces character sequences without making any independence assumptions between the characters.
  - Modules:
    - Listener - encodes information about the input audio
    - Attend - tells Speller what parts of input are relevant during the current decoding step
    - Speller- decodes the latent representation into transcription

https://arxiv.org/abs/1508.01211



15

Lecture 4, PDF p. 15 — pyramidal listener, attention context, autoregressive speller

Listener는 음향 frame을 인코딩하고 Attend는 가중합 context를 만든다. Speller는 context와 이전 token으로 다음 token을 생성한다.

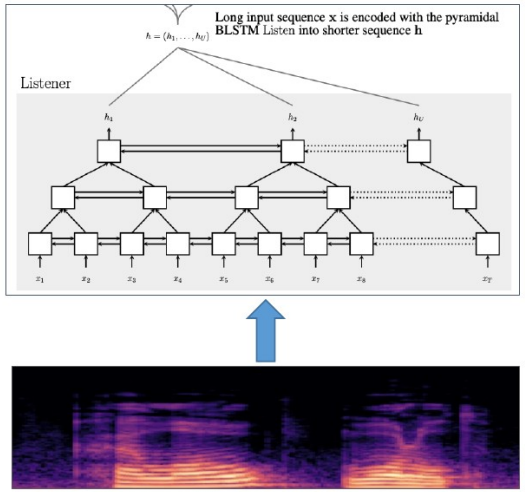
학습은 정답 prefix를, 추론은 생성 prefix를 쓰므로 오류가 누적될 수 있다.

LAS: Why the Pyramid?

Listener가 이웃한 시간 표현을 묶어 attention이 읽을 길이를 줄인다.

Listen, Attend and Spell (LAS)

- In fact, we can use arbitrary encoder (for example Conformer), but that was in 2015
- Thus, we use LSTM in the encoder and a pyramid structure for subsampling since we need to reduce the time dimension:
  - RNN can't handle very long sequences
  - Reduce the time dimension for Attend for selecting the most relevant information
  - Better convergence in much shorter time
- 8x time reduction (4x in the image)



https://arxiv.org/abs/1508.01211

16

Lecture 4, PDF p. 16 — 4× reduction in the diagram; 8× in the described LAS model

그림은 인접 frame을 이어 붙여 상위 recurrent layer에 보내는 흐름이다. 두 번 절반으로 줄이면 T/4가 된다.

원문의 모델 설명은 8배 축소, 그림은 4배 축소다. 입력 정보가 공짜로 보존되는 것은 아니며 세부 시간 해상도와 계산량의 교환이다.

LAS: Greedy Can Miss the Best Text

첫 단계 P(A)=0.6, P(B)=0.4. 다음 확률은 선택한 prefix마다 다르다.

Sequence	Conditional probability	Sequence score
AC	P(C|A)=0.51	0.6 × 0.51 = 0.306
AD	P(D|A)=0.49	0.6 × 0.49 = 0.294
BC	P(C|B)=0.90	0.4 × 0.90 = 0.360
BD	P(D|B)=0.10	0.4 × 0.10 = 0.040

Greedy는 A → C를 고른다. Beam 2는 B도 남겨 최종적으로 더 높은 BC를 찾는다.
설명용으로 길이를 2, 그 뒤 EOS 확률을 1로 둔다. CTC처럼 여러 정렬을 합한 예제는 아니다.

LAS Summary: Strength and Cost

LAS는 출력 문맥을 얻지만, 매 token의 생성과 attention 계산이 순차적으로 이어진다.

CTC와 달리 이전 문자를 조건으로 다음 문자를 예측한다. Attention은 현재 출력에 필요한 음향 위치를 선택하며, frame별 정답 시각을 직접 요구하지 않는다.

기본 LAS는 전체 입력을 보는 Listener·attention 때문에 streaming에 적합하지 않다. 제한된 문맥이나 monotonic attention을 쓰는 변형과 구별해야 한다.

음향 정렬의 불확실성을 모두 경로 합으로 푸는 CTC와 달리 LAS는 context를 통해 정렬을 학습한다. 출력 오류 누적과 긴 입력의 비용이 남는다.

RNN-T : Audio and Output History

RNN-T는 CTC의 시간 진행과 LAS의 출력 문맥을 함께 다루려는 모델이다.

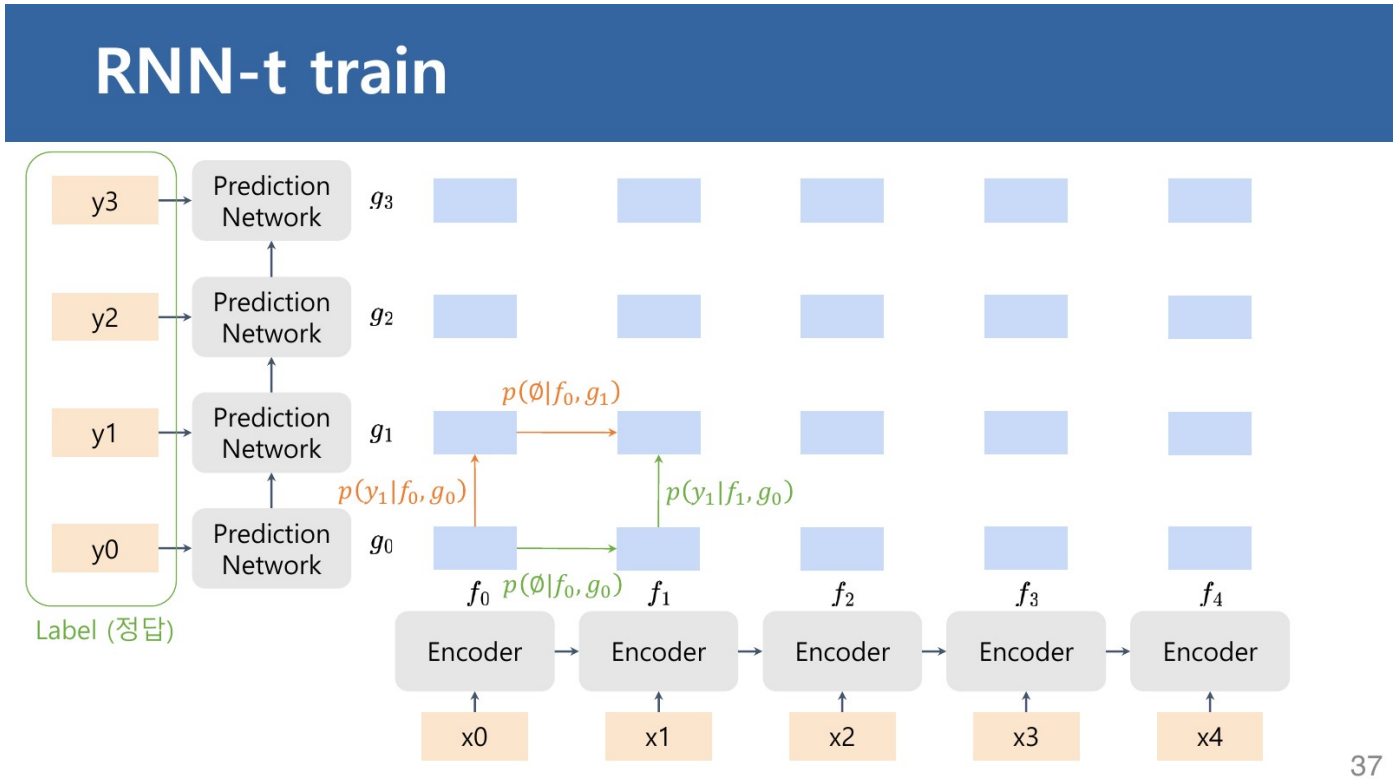
Encoder f(t)는 입력 음향을, prediction network g(u)는 지금까지 출력한 u개 token을 표현한다. Joint network가 두 표현을 합쳐 token·blank 분포를 만든다.

Blank를 내면 다음 음향 시점으로 간다. Token을 내면 음향 시점은 유지하고 출력 이력만 갱신한다. 하나의 시점에서 여러 token을 낼 수 있다.

Prediction network는 출력 prefix를 읽는다. 하지만 별도 텍스트로 학습한 외부 LM과 동일한 물체는 아니다. Streaming 역시 encoder 문맥 조건에 달린다.

RNN-T Lattice

RNN-T는 acoustic time과 output prefix를 두 축으로 둔다.



Blank는 (t,u)에서 (t+1,u)로 시간을 진행한다. Token은 (t,u+1)로 출력 길이를 늘린다.

Prediction network가 이전 출력 이력을 표현하므로 같은 frame에서도 여러 token을 낼 수 있다.

Lecture 4, PDF p. 37 — encoder and prediction-network axes in RNN-T training

RNN-T: Walk Through Inference

원문 cat 예제는 blank가 시간축, 문자가 출력축을 움직이는 모습을 보여 준다.

빈 prefix에서 x_0의 blank를 선택하면 x_1로 간다. x_1에서 c를 출력하면 prefix는 c가 되지만 x_1에 머문다. 다음 blank가 x_2로 보낸다.

x_2에서 a를, 다시 같은 x_2에서 t를 출력한다. 그 뒤 blank들이 남은 음향 시점을 소비한다. 'cat'은 3개 token이지만 3개 frame과 일대일 대응하지 않는다.

RNN-T의 반복 문자는 CTC처럼 병합하지 않는다. l 다음 l을 실제로 출력하면 ll이다. CTC의 collapse 규칙을 이 격자에 적용하면 안 된다.

RNN-T: Train Over All Alignments

정답 문자열은 알아도 각 token이 어느 시점에서 나올지는 모르므로, 가능한 정렬을 합한다.

학습에서는 정답 prefix별 g(u)와 음향 시점별 f(t)를 조합한다. 각 (t,u)의 softmax에서 blank 확률과 다음 정답 token 확률을 읽는다.

한 정렬 경로는 선택한 전이 확률의 곱이다. 정답을 완성하고 입력을 소비하는 모든 단조 경로의 확률을 합해 P(y|X)를 얻는다.

Loss=−ln P(y|X). 정답 token 시각을 하나로 고정하거나, greedy로 고른 한 경로만 학습하는 것이 아니다. Forward DP가 중복 부분 경로 계산을 줄인다.

RNN-T: A Forward-DP Cell

같은 칸에 도달하는 두 방향의 확률을 더한다.

α (t,u)는 t개 음향 시점을 지나 u개 정답 token을 낸 경로의 확률 합이다. 왼쪽에서는 blank, 아래에서는 정답 token을 내며 들어온다.

내부 칸: α (t,u)=α (t−1,u)b(t−1,u)+α (t,u−1) l (t,u−1). b는 blank, l 은 다음 정답 token의 조건부 확률이다.

예: 0.20×0.30 + 0.10×0.40 = 0.10. 두 항은 마지막 전이가 서로 다른 경로 묶음이다. 서로 배타적이라 합하며, 한 경로를 두 번 세지 않는다.

RNN-T DP: Start, Boundary, Finish

T개 frame을 모두 소비하고 U개 정답 token을 내면 종료한다.

시작 $\alpha(0,0)=1$, 불가능한 상태는 0이다. $t<T$에서 blank는 $(t,u)\to(t+1,u)$, 다음 정답 token은 $u<U$일 때 $(t,u)\to(t,u+1)$로 이동한다.

첫 열 $\alpha(t,0)=\alpha(t-1,0)\,b(t-1,0)$. 첫 행 $\alpha(0,u)=\alpha(0,u-1)\,l(0,u-1)$. 들어오는 방향이 하나뿐이므로 그 전이 확률만 곱한다.

$t=T$에서는 token 전이가 없다. 마지막 frame에서 blank로 들어온 $\alpha(T,U)$가 $P(y|X)$다. 따라서 존재하지 않는 $f(T)$를 계산하거나 마지막 blank를 다시 곱하지 않는다.

RNN-T Properties: Memory Cost

입력 길이 T와 정답 길이 U의 조합마다 joint 계산이 필요하다.

RNN-t properties

- streamable and context dependent
- produce alignment (weak)
- it is difficult to compute efficiently
  - calculation of the joint module is hard
    BS = 32 ; T (after 2x stride) = 800, U (with character encoding) = 400-450 tokens, V = 28. Let the hidden dimension of the Joint model be 640.
    - Hidden = 32×800×450×640×4 = 29.49 Gb (fp32)
      Joint = 32×800×450×28×4 = 1.290 Gb (fp32)
    - This is just for the forward pass. We need to double this memory to store gradients.
  - BS=32 ; T (after 8x stride) = 200, U (with sub-word encoding) = 100-180 tokens, Vocabulary size V = 1024.
    - Hidden = 32×200×150×640×4 = 2.45 Gb (fp32)
      Joint = 32×200×150×1024×4 = 3.93 Gb (fp32)

42

Lecture 4, PDF p. 42 — joint-grid memory; GB (bytes), not Gb (bits)

원문은 batch·T·U·hidden 차원을 곱해 중간 tensor의 크기를 계산한다. FP32 한 원소는 4 byte다.

29.49라는 값의 단위는 Gb가 아니라 약 29.49 GB다. 이는 특정 tensor의 크기이며 전체 학습 메모리와 같지 않다.

RNN-T: Length and Vocabulary Trade-off

T와 U를 줄여도 vocabulary V가 커지면 출력 tensor는 커질 수 있다.

Tensor	Character setup	Subword setup
B, T, U	32, 800, 450	32, 200, 150
Hidden width H	640	640
B × T × U × H × 4	29.49 GB	2.46 GB
B × T × U × V × 4	1.29 GB (V=28)	3.93 GB (V=1024)

U=150은 원문 범위 100–180의 계산 예다. FP32·십진 GB이며 전체 학습 메모리는 아니다.
Subsampling·subword는 길이를 줄이지만, 큰 어휘의 softmax 비용도 함께 비교해야 한다.

CTC, LAS, and RNN-T

세 모델은 같은 정렬 문제를 서로 다른 확률 구조로 푼다.

CTC vs. LAS vs. RNN-t

Method	CTC	LAS	RNN-t
Streaming	yes	no	yes
Context	no	yes	yes
Argmax Complexity	O(enc + dec)	O(enc + T * dec)	O(enc + T * dec)
Beam Search Complexity	O(enc + dec + T * bs)	O(enc + T * bs * dec)	O(enc + T * bs * dec)

CTC는 출력 이력 없이 frame 경로를 합한다. LAS는 attention과 autoregressive prefix를 쓴다. RNN-T는 prefix와 시간축 경로를 함께 합한다.

Streaming 여부는 encoder 문맥과 실제 추론 설정까지 확인해야 한다.

Lecture 4, PDF p. 44 — lecture comparison of CTC, LAS, and RNN-T

Read the Comparison: Context and Time

원문 표의 Context는 출력 token 이력, Streaming은 입력을 기다리는 조건에 관한 말이다.

CTC의 Context=no는 음향 문맥이 없다는 뜻이 아니다. Encoder는 문맥을 볼 수 있지만 분포가 이전에 선택한 출력 token에 직접 의존하지 않는다.

LAS와 RNN-T는 출력 prefix를 사용한다. 기본 LAS는 전체 입력 attention을 쓰고, RNN-T는 blank를 통해 시간을 진행해 streaming 구조로 만들기 쉽다.

CTC·RNN-T도 양방향 전체 문맥 encoder면 미래 입력을 기다린다. 표의 yes/no는 기본 구조의 성질이지 모든 구현에 대한 무조건적 보장이 아니다.

Read the Comparison: Decoding Cost

복잡도 표의 핵심은 '출력 이력마다 decoder를 다시 계산하는가'이다.

CTC는 encoder가 frame 분포를 만들면 후보 경로들이 그 분포를 공유한다. Beam을 늘리면 경로 누적·병합 계산은 늘지만 별도 AR decoder가 생기지는 않는다.

LAS·RNN-T는 prefix마다 상태가 다르므로 후보별 decoder 또는 joint 계산이 필요하다. Beam을 늘릴 때 확률 누적 외에 신경망 계산 부담도 커질 수 있다.

원문의 enc·dec·T·bs 식은 개략 비교다. 실제 비용은 입력 T, 출력 U, 어휘 V, beam B, cache와 batching에 달린다. 연도별 WER만으로 구조의 우열도 단정하지 않는다.

Read the Results: WER and Conditions

WER은 낮을수록 좋지만, 다른 학습 조건의 결과를 구조 하나의 효과로 볼 수는 없다.

RNN-t Quality

Method	Year	WER (test-clean)	WER (test-other)
Human	~ 200 000 b.c.	5.83	12.69
Deep Speech 2	2015	5.15	12.73
LAS	2015	----	----
LAS without SpecAugment*	2019	3.2	9.8
RNN-t	2020	**2.1**	**4.3**

*https://arxiv.org/abs/1904.08779

43

Lecture 4, PDF p. 43 — reported WER values under different years and training settings

표는 DS2, LAS, RNN-T의 서로 다른 연도 결과다. Test-clean과 test-other도 나눠 읽어야 한다.

L3는 DS2에 11.9k시간·35M parameter·16 GPU·3–5일을 제시한다. 데이터와 자원 차이까지 봐야 하며 이 표만으로 구조의 우열을 확정할 수 없다.

Language Models: Why Add One?

소리가 비슷한 후보를 언어 문맥으로 구별한다.

원문의 'go two a movie'와 'go to a movie'는 소리만으로 헷갈릴 수 있다. LM은 문장 문맥에서 어느 token 열이 자연스러운지 점수를 준다.

설명용 λ =1, 길이 보정 없음: 0.21×0.01=0.0021, 0.19×0.60=0.114. 음향 점수는 낮아도 LM 결합 점수에서 'to' 후보가 앞설 수 있다.

보통 log P_ASR(y|X)+λ log P_LM(y)+길이 보정을 사용한다. 이는 재정규화된 확률이 아닌 탐색 점수이며, λ 가 너무 크면 음향 증거를 압도할 수 있다.

Rescoring vs. Shallow Fusion

언어모델을 탐색이 끝난 뒤 쓰는지, 후보를 남길 때부터 쓰는지가 다르다.

Second-pass rescoring: ASR이 만든 N-best 후보를 LM으로 다시 정렬한다. 첫 탐색에서 이미 버린 문장은 되살릴 수 없다. 비싼 LM도 제한된 후보에 쓸 수 있다.

Shallow fusion: beam 확장 단계마다 ASR log 점수와 LM log 점수를 더한다. LM이 중간 후보의 생존에 영향을 주지만 반복 호출 비용이 든다.

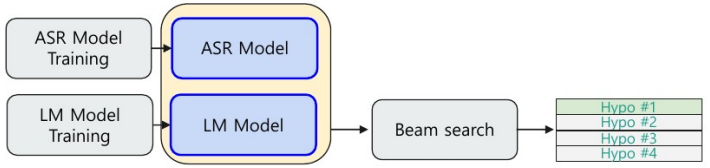
두 방식 모두 ASR·LM을 따로 학습할 수 있다. 단순 점수 결합은 내부 feature를 함께 학습하는 deep·cold fusion과 구분한다.

Deep Fusion vs. Cold Fusion

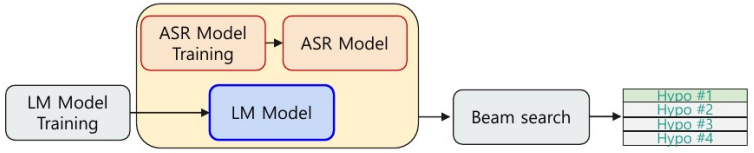
그림의 차이는 사전학습 LM을 ASR 학습에 언제 연결하는가이다.

DL-based ASR

- Language models (LM) with LAS and RNN-t
  - Deep Fusion:



  - Cold Fusion:



48

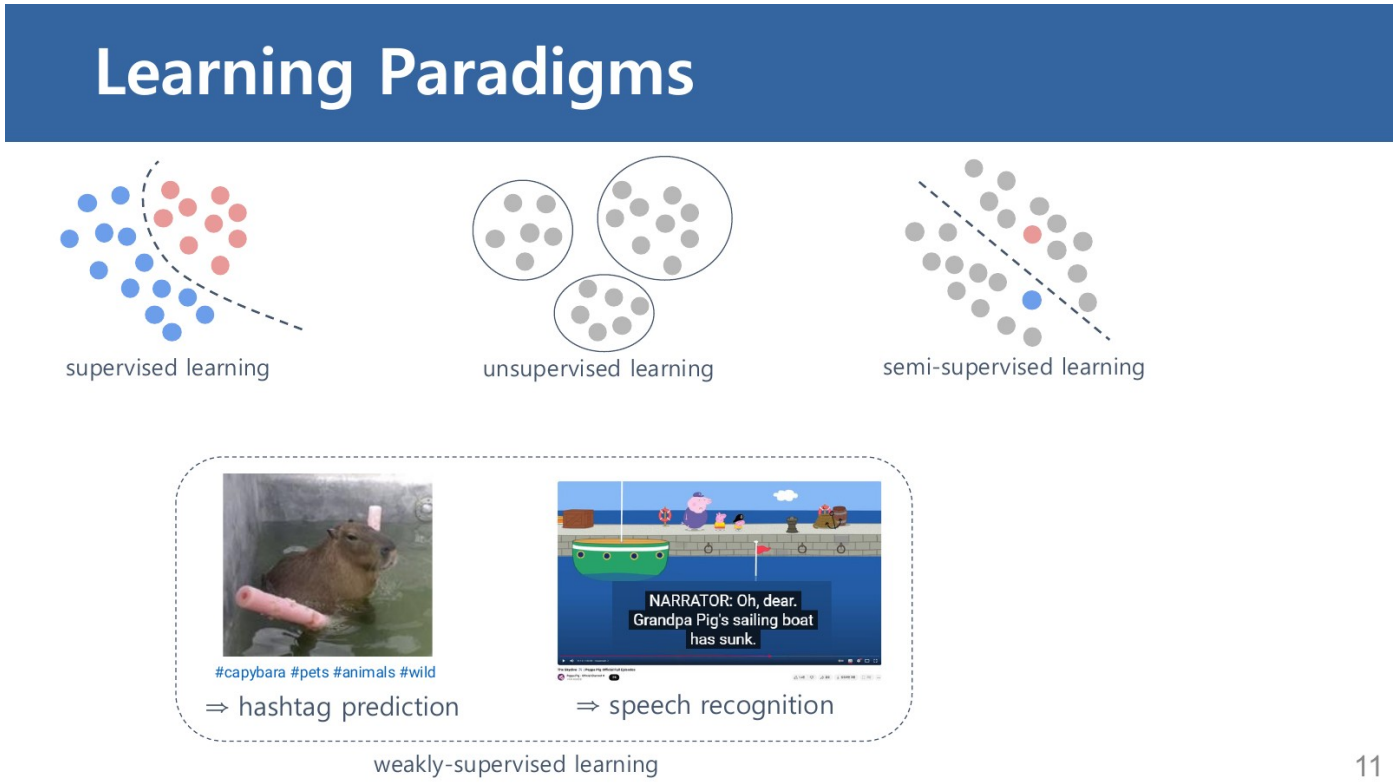
Lecture 4, PDF p. 48 — separately trained models vs. LM-conditioned ASR training

위쪽 deep fusion: 따로 학습한 ASR·LM의 내부 표현을 연결하고 결합부를 학습한다. 단순 log 점수 덧셈이 아니다.

아래쪽 cold fusion: 사전학습 LM을 ASR 학습 단계부터 활용한다. ASR이 LM에 맡길 언어 정보와 음향 정보를 함께 분담하도록 배운다.

Learning Paradigms: Read the Source

그림은 같은 데이터를 다루더라도 정답 신호의 출처가 다를 수 있음을 보여 준다.



Supervised는 명시적 정답, unsupervised는 별도 정답 없는 구조 탐색이다. Semi-supervised는 labeled·unlabeled 자료를 함께 활용한다.

아래 hashtag 예시는 weak supervision이다. 자료와 연결된 불완전한 신호를 이용하며, 입력 자체에서 target을 만드는 SSL과 구분한다.

Lecture 5, PDF p. 11 — learning paradigms and weak supervision

Learning Paradigms: The Target Source

무엇을 정답으로 삼는지 먼저 물으면 학습 방식이 구별된다.

Paradigm	Supervision	Speech example
Supervised	사람이 준 정답	음성–전사 쌍으로 ASR
Unsupervised	명시적 정답 없음	음성 표현의 군집화
Semi-supervised	정답 있음 + 없음	전사 자료 + pseudo-label
Self-supervised	입력에서 target 구성	가린 음성 표현 예측

이 분류는 완전히 배타적이지 않다. SSL 사전학습 뒤 supervised fine-tuning을 할 수 있다.
학습 파이프라인 전체와 개별 학습 목표의 분류를 혼동하지 않는다.

Weak Supervision Is Not No Supervision

전사가 완벽하지 않아도 음성과 연결된 텍스트를 학습 신호로 쓸 수 있다.

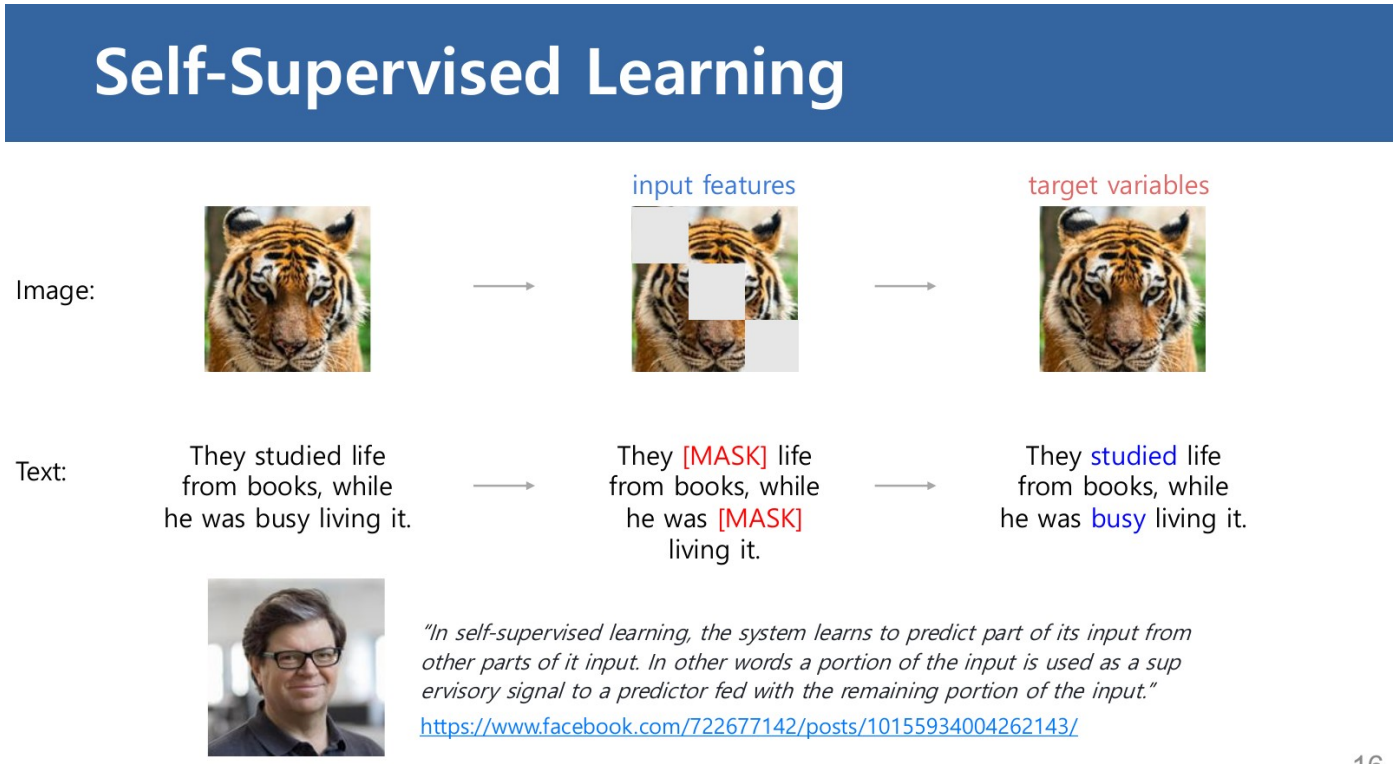
원문은 Whisper의 인터넷 음성–텍스트 680k시간과 MMS의 1,000개 이상 언어를 소개한다. 대규모로 연결된 자료를 활용한다는 점이 핵심이지 두 수치의 단위가 같은 것은 아니다.

자료가 크다고 label이 정확해지는 것은 아니다. 잡음 있는 대응, 도메인 편향, 언어별 차이를 점검해야 한다. MMS 전체 학습 단계를 모두 weak-only로 분류하지 않는다.

SSL의 핵심은 별도 전사 대신 입력에서 target을 만드는 것이다. Weak supervision은 불완전하더라도 외부에서 연결된 label 신호가 있다는 차이가 있다.

Self-Supervision: Input Becomes Target

학습할 답이 없는 것이 아니라, 입력 자료에서 답을 구성한다.



가운데는 일부를 가린 입력, 오른쪽은 원래 자료에서 얻은 target이다. 모델은 보이는 문맥으로 가린 부분을 예측한다.

정답을 통째로 입력에 노출하면 복사로 풀릴 수 있다. 어떤 부분을 가리고, 무엇을 맞히게 하느냐가 학습되는 표현을 결정한다.

Lecture 5, PDF p. 16 — masked input and targets derived from the original data

SSL: Objective, Benefit, and Limit

사전학습에서 배운 표현을 적은 전사 자료로 ASR에 적응시킨다.

입력 변형 → target 구성 → 모델 예측 → loss 계산이 사전학습 문제다.
텍스트에서는 가린 token 또는 다음 token을 예측하지만 모든 SSL 방식이 같지는 않다.

원문의 빠른 수렴·적은 label·대규모 표현은 기대 효과다. Task와 자료가 맞을 때 도움이 되며, 도메인 차이·잡음·부적절한 target이면 개선을 보장하지 않는다.

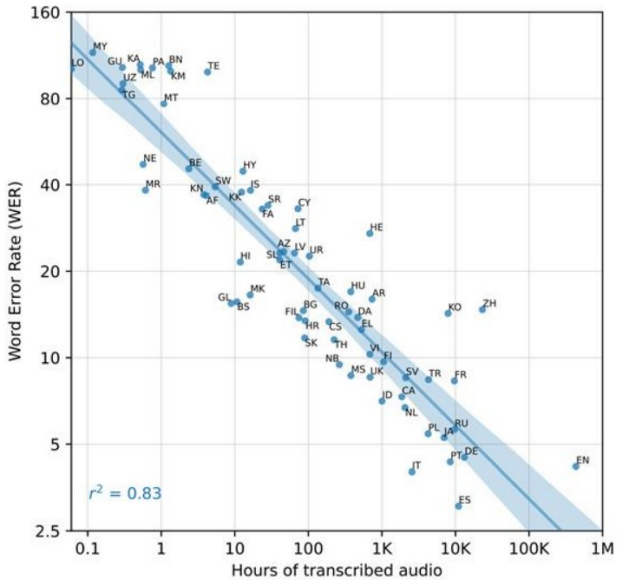
사전학습 target을 잘 맞히는 것과 문자를 정확히 전사하는 것은 다르다. ASR fine-tuning에서는 전사와 CTC 같은 task loss로 목적을 다시 맞춘다.

SSL Motivation: Read the Data Plot

전사 자료가 적을수록 어려워지는 문제를 보여 주는 그래프다. SSL의 인과 효과를 증명하는 그래프는 아니다.

Self-Supervised Learning

- fast convergence
- better convergence (data)
- low-resource
- large-scale models (data)
- foundation model



17

가로는 전사 음성 시간, 세로는 WER이며 둘 다 로그 눈금이다. 언어별 점에서 자료량이 클수록 WER이 낮아지는 경향을 본다.

r^2=0.83은 이 관계의 회귀 적합도다. 자료량만이 원인이라는 뜻도, SSL을 쓰면 WER이 83% 감소한다는 뜻도 아니다.

Lecture 5, PDF p. 17 — transcribed hours vs. WER; association is not a causal SSL gain

Speech SSL: Why Targets Are Difficult

음성 파형은 연속값이며, 처음부터 단어 사전처럼 나뉘어 있지 않다.

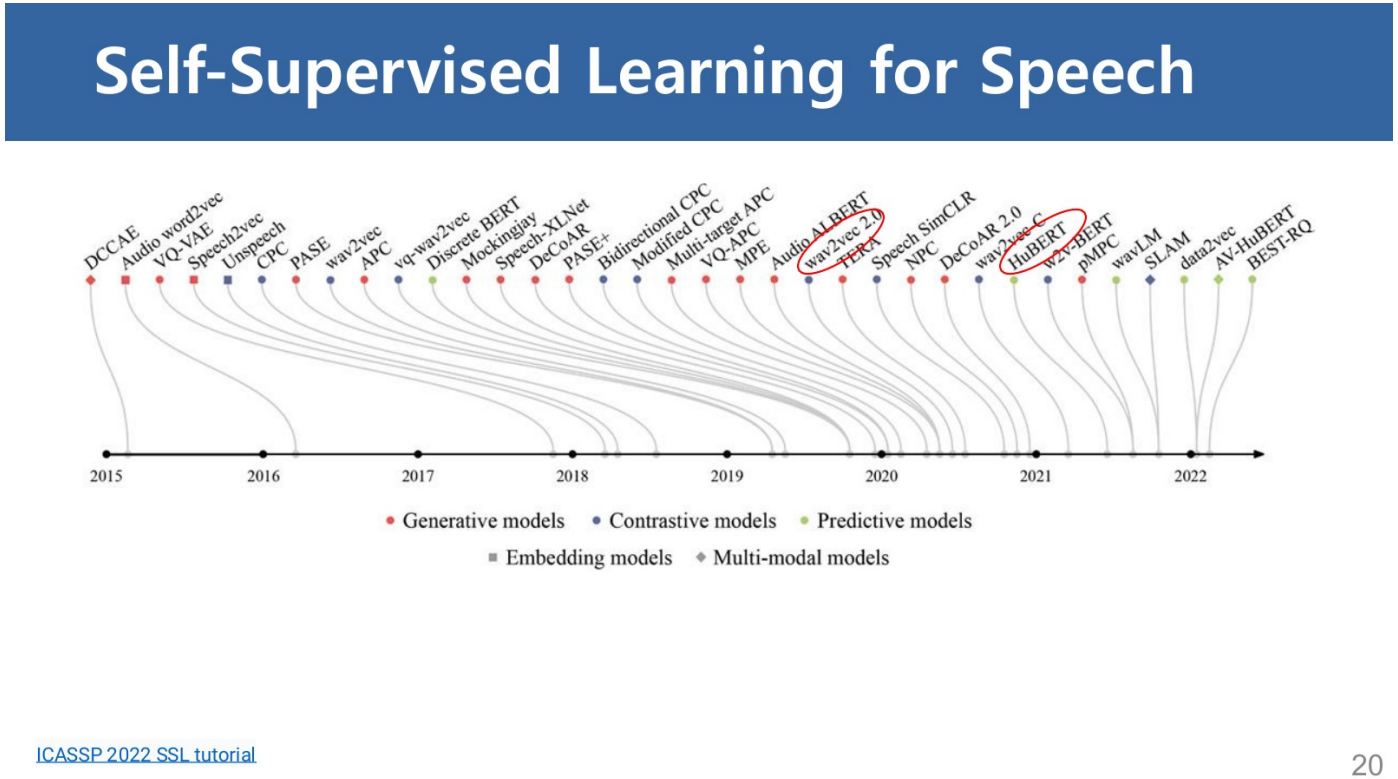
16 kHz mono 음성 1초는 16,000개 sample이다. 각 값은 단어 token이 아니며 화자·잡음·위상 같은 변화도 포함한다. 무엇을 보존할지 선택해야 한다.

원파형 복원만 잘하면 언어 내용을 반드시 잘 이해하는 것은 아니다. 문맥 예측, 후보 구별, 표현의 군집화 등 서로 다른 target 설계가 가능하다.

따라서 음성 SSL은 입력 표현·가리는 방식·target·loss를 함께 본다. 다음 wav2vec 2.0에서는 연속 표현에서 이산 target을 만드는 방법으로 이어진다.

Speech SSL Timeline: Read the Families

계보의 핵심은 모델 이름보다 학습 신호를 만드는 방식이다.



ICASSP 2022 SSL tutorial

20

범례는 generative·contrastive·predictive 등 목표의 계열을 나눈다. 복원할지, 후보를 구별할지, target label을 예측할지가 서로 다르다.

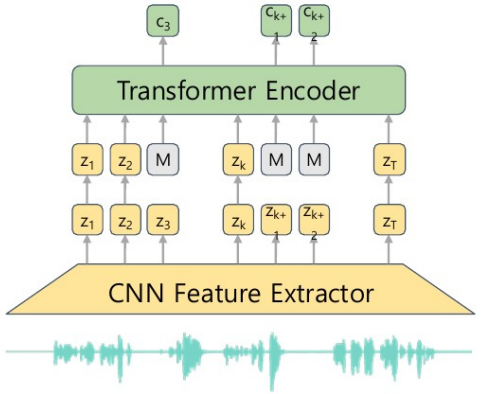
이 그림은 2015–2022년 흐름이며 성능 순위표가 아니다. 강의는 여기서 wav2vec 2.0과 HuBERT를 골라 target 설계를 비교한다.

Lecture 5, PDF p. 20 — historical speech SSL families, not a performance ranking

wav2vec 2.0 Context

가린 latent 위치를 주변 음성 문맥으로 표현한다.

wav2vec2.0



24

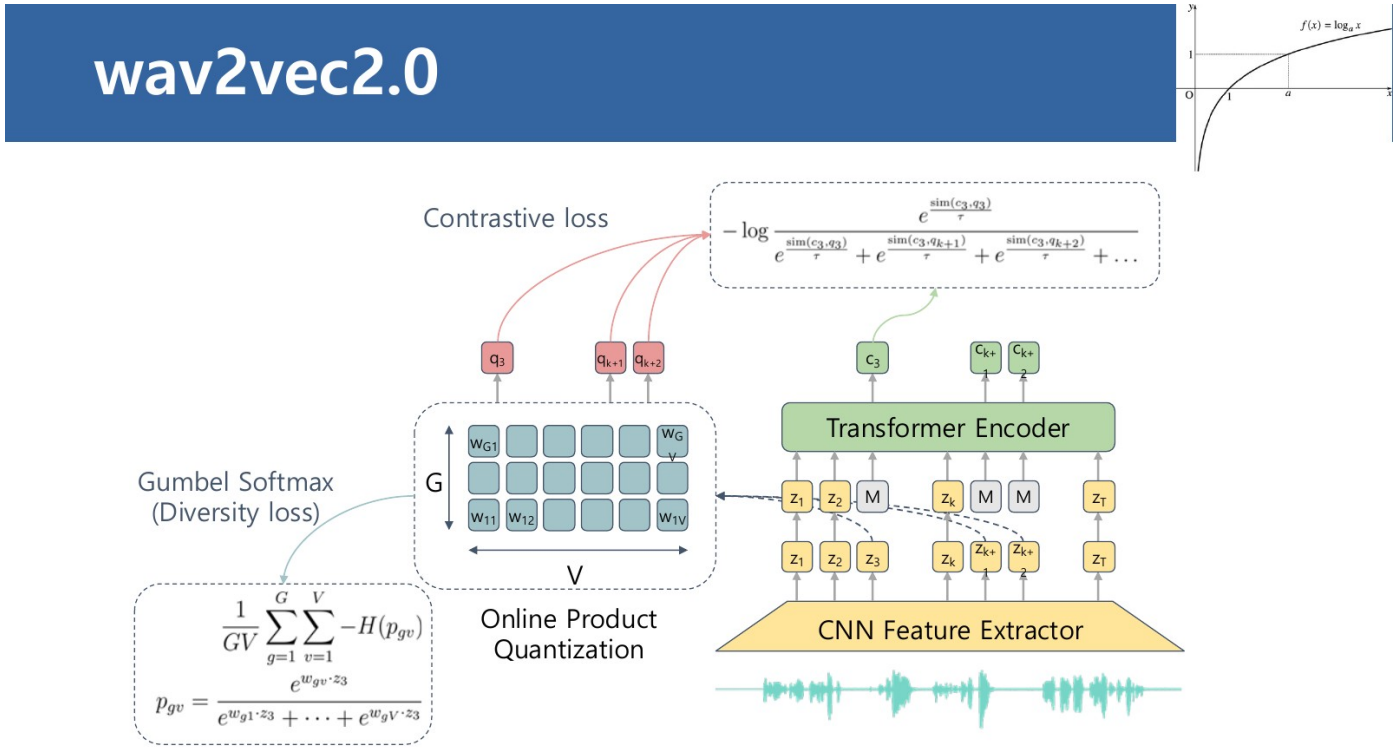
CNN이 waveform을 z_t로 바꾸고, 일부 위치를 mask embedding으로 치환한 뒤 Transformer가 c_t를 만든다.

Masking은 spectral mask로 음원을 복원하는 연산이 아니라 사전학습 문제를 만드는 절차다.

Lecture 5, PDF p. 24 — CNN latent sequence, masked positions, Transformer context

wav2vec 2.0 Targets

Context c_t는 같은 위치의 quantized target q_t를 distractor보다 잘 골라야 한다.



31

가리지 않은 z_t는 product quantizer를 거쳐 q_t가 된다. Contrastive loss는 positive와 후보를 구별한다.

Diversity objective는 일부 code entry만 쓰는 collapse를 줄인다. q_t는 문자 label이 아니다.

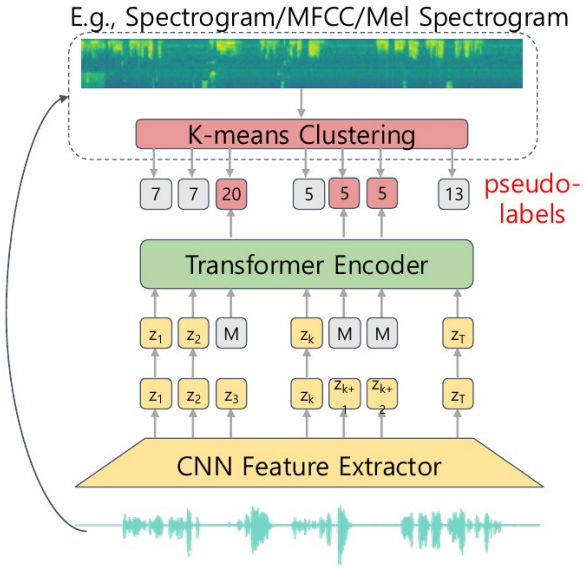
Lecture 5, PDF p. 31 — online quantization, contrastive loss, diversity loss

HuBERT Hidden Units

HuBERT는 offline clustering으로 만든 ID를 가린 위치에서 예측한다.

HuBERT

- HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units (Meta, 2021)
- **H**idden **U**nit **BERT**



46

첫 target은 MFCC 등의 feature를 k-means로 묶어 만든다. 7·20·5는 cluster ID이며 음소 이름이나 크기 순서가 아니다.

이전 HuBERT layer를 다시 clustering해 target을 개선할 수 있다.

Lecture 5, PDF p. 46 — offline k-means pseudo-labels and masked prediction

Final Recap

표현, encoder, 정렬 목표는 서로 다른 설계 축이다.

Question	Method	Answer
무엇을 보존?	STFT/Mel	time-frequency / compressed bands
어떻게 정렬?	CTC/LAS/RNN-T	paths / attention / 2-D lattice
어떻게 인코딩?	DS2/Conformer	RNN / attention+conv
target은 어디서?	wav2vec 2.0/HuBERT	quantizer / clustering

FFT는 DFT 계산법이고, Mel은 지각적 축약이다. Conformer는 encoder이며 CTC는 정렬 목표다.
wav2vec 2.0은 online quantized target, HuBERT는 offline cluster target을 쓴 뒤 labeled ASR로 fine-tuning 한다.